Dynamo on Intel XPU/GPU: Kubernetes Deployment Whitepaper

Contents

Dynamo on Intel XPU — Deployment Whitepaper

Scope: This document covers deploying Dynamo on Intel XPU (GPU) hardware in Kubernetes, using the official Intel XPU templates shipped in the Dynamo repository. It replaces informal/outdated internal notes on Dynamo + Intel XPU with a single source of truth, driven directly from the current state of the dynamo repository.

All content below is derived from reading the actual repository files (YAML templates, README.md files, docs/kubernetes/*). Nothing has been executed against a live Intel XPU Kubernetes cluster yet — see the Status line in each chapter, and the companion tracking.md file for the current per-case validation status. Validate on real hardware before treating any command as final.

API version note: Chapter 1's 8 templates use the nvidia.com/v1alpha1 CRD schema (services: map) — this is what the repo ships for all 8 Intel XPU patterns today. Dynamo is mid-migration to a newer nvidia.com/v1beta1 schema (components: list, a real structural change, not just a version bump), but **XPU coverage under v1beta1 is only partial**: just 2 of the 8 patterns (agg, disagg) have been ported so far — see the companion document to_fi.md (shipped alongside this whitepaper, not included in this compiled PDF) for the full breakdown. If you're on a newer Dynamo Platform release that expects v1beta1, check deploy/v1beta1/xpu/ for the latest state before assuming all 8 patterns are available there.

Why this document exists / scope note

Dynamo's Intel XPU support is concentrated in a single directory: examples/backends/vllm/deploy/ This directory ships **8 flat DRA (Dynamic Resource Allocation) YAML templates** covering combinations of serving pattern (aggregated / disaggregated) × optional add-ons (tracing, KV-aware routing, planner-based autoscaling). There is also one Intel XPU example under examples/global_planner/.

A repo-wide search (grep -r xpu examples/ docs/ recipes/ across the dynamo repository) turns up **no Intel XPU-specific content** under most other vLLM-backend templates, the SGLang backend, the TensorRT-LLM backend, and most of recipes/. It **does** turn up one Intel XPU recipe under recipes/qwen3-vl-32b-fp8/ — a heterogeneous Intel XPU (encode) + NVIDIA GPU (decode) disaggregation, written up in full in Chapter 3. Chapter 4 distinguishes templates that look **feasible but simply haven't been templated for Intel XPU yet** (LoRA, GAIE, multi-node disaggregation, GMS/failover — all with no hard hardware blocker found) from the one **genuine NVIDIA-only dependency** found (TensorRT-LLM's proprietary compiler).

Bottom line: as of this writing, if you want to run Dynamo on Intel XPU, your options are the 8 templates in Chapter 1, the Global Planner example in Chapter 2, and the heterogeneous Qwen3-VL recipe in Chapter 3. Chapter 4 catalogs real candidates for future Intel XPU work

(LoRA, GAIE, multi-node disagg, GMS/failover, SGLang) versus the one genuine NVIDIA-only dead end confirmed so far (TensorRT-LLM).

Table of Contents

Status labels used below: **Verified** (ran hands-on and confirmed working) / **In Progress** (documented from official sources, not yet run hands-on) / **N/A** (not applicable — no Intel XPU support exists upstream)

`tracking.md` and `to_fi.md` in this directory are separate, continuously-updated tracking documents — useful when browsing the repository directly, but they are not chapters of this whitepaper and are not included in the compiled PDF/Word export.

Chapter 0 — Common Installation

All 8 templates in Chapter 1, plus the Global Planner example in Chapter 2, share the same cluster-level prerequisites. Do this section once per cluster.

0.1 Cluster Requirements

- **Kubernetes v1.34+** with the DRA (Dynamic Resource Allocation) `resource.k8s.io/v1` API enabled (DRA graduated to GA/stable behavior tracked by the templates; older DRA API versions such as `v1alpha3`/`v1beta1` are not compatible with the `apiVersion: resource.k8s.io/v1` used by every current template).
- Intel XPU (GPU) nodes, with the Intel GPU kernel driver installed on each node.
- **Intel resource drivers for Kubernetes** installed, exposing a `DeviceClass` named `gpu.intel.com` (see 0.1.1 below). Verify with:

```
kubectl get deviceclass gpu.intel.com
```

- Sufficient node resources for the model under test — the default model across all Chapter 1/2 templates is the small `Qwen/Qwen3-0.6B`, chosen deliberately by the upstream repo to keep XPU examples runnable on a single GPU.

0.1.1 Install the Intel GPU DRA Driver

Not part of the dynamo repo itself — this is a cluster-level, vendor-provided component that must be installed once per cluster before any of the templates in this whitepaper. Install via its published Helm chart:

```
helm install \
  --namespace intel-gpu-resource-driver \
  --create-namespace \
  intel-gpu-resource-driver oci://ghcr.io/intel/intel-resource-drivers-for-
 ↪  kubernetes/intel-gpu-resource-driver-chart
```

Verify the driver is up and the `gpu.intel.com` `DeviceClass` and per-node `ResourceSlice` objects were published:

```
kubectl get pods -n intel-gpu-resource-driver
kubectl get deviceclass gpu.intel.com
kubectl get resourceslices
```

See the driver's own GPU documentation for container-runtime CDI requirements (CRI-O 1.23+ or Containerd 1.7+ with CDI enabled) and alternative install methods (kustomize, Node Feature Discovery-scoped rollout).

0.2 Install the Dynamo Kubernetes Platform

Follow `docs/kubernetes/installation-guide.md` in the dynamo repo. Summary of the steps that apply regardless of accelerator vendor:

```
export NAMESPACE=dynamo-system
export RELEASE_VERSION=<dynamo-release-version>   # match a version from the
 ↪  repo's releases

helm fetch https://helm.ngc.nvidia.com/nvidia/ai-dynamo/charts/dynamo-
 ↪  platform-${RELEASE_VERSION}.tgz
helm install dynamo-platform dynamo-platform-${RELEASE_VERSION}.tgz \
  --namespace ${NAMESPACE} \
  --create-namespace
```

> **Note**: the `dynamo-crds` Helm chart is deprecated as of v1.0.0 — CRDs are now installed and managed by the Dynamo Operator itself as part of the `dynamo-platform` chart above. No separate CRD install step is needed on current releases.
>
> **Intel XPU note**: unlike NVIDIA deployments, you do **not** need the NVIDIA GPU Operator. Skip any GPU-Operator-specific step in the installation guide; the Intel device plugin/DRA driver from step 0.1 takes its place.

Confirm the operator is healthy:

```
kubectl get pods -n ${NAMESPACE}
kubectl get crd | grep dynamo
```

0.3 Build an Intel XPU vLLM Runtime Image

None of the 8 templates work with the stock `vllm-runtime` image — worker containers require a dedicated `vllm-runtime-xpu` image built with `VLLM_TARGET_DEVICE=xpu`. Build it from the repo's `container/` tooling:

```
cd dynamo/container
python render.py --framework=vllm --device=xpu --target=runtime
docker build -t <your-registry>/vllm-runtime-xpu:my-tag \
  -f vllm-runtime-xpu-amd64-rendered.Dockerfile .
docker push <your-registry>/vllm-runtime-xpu:my-tag
```

See `container/README.md` for the full, up-to-date build instructions and base image options. Substitute `<your-registry>/vllm-runtime-xpu:my-tag` for every occurrence of `nvcr.io/nvidia/ai-dynamo/vllm-runtime-xpu:my-tag` in the templates below, either by editing the YAML or by using `envsubst` with an image-name environment variable if your fork of the template parameterizes it.

The Frontend and Planner components do **not** need the XPU image — they run on the default `vllm-runtime` / `dynamo-planner` images (CPU-only control-plane components).

0.4 HuggingFace Token Secret

Every template references `envFromSecret: hf-token-secret`. Create it once per namespace:

```
export HF_TOKEN=<your-huggingface-token>
kubectl create secret generic hf-token-secret \
  --from-literal=HF_TOKEN=${HF_TOKEN} \
  -n ${NAMESPACE}
```

0.5 Optional: `securityContext` for Render/Video Group Access

Every worker container block in every template includes a commented-out `securityContext` snippet:

```
# NOTE: Uncomment if your environment requires specific group access
# securityContext:
#   runAsUser: 1000
#   runAsGroup: 1000
#   supplementalGroups:
#     - 44   # render group
#     - 991  # video group
```

Uncomment and adjust the GIDs if your cluster's Pod Security Admission policy or your node image's `/dev/dri` device permissions require the container process to belong to specific `render`/`video` groups to access the Intel GPU. GIDs 44/991 are the upstream template's defaults and may differ from your actual node OS image — check `getent group render video` on a node.

1.1 Aggregated (`agg_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: The simplest pattern — a single `VllmDecodeWorker` role handles both prefill and decode for every request (no P/D separation). Good starting point to validate that DRA GPU

allocation and the XPU runtime image work at all, before adding routing/tracing/disaggregation on top.

Prerequisites: Chapter 0 complete.

Deployment Steps:

```
export NAMESPACE=my-ns
kubectl create namespace ${NAMESPACE} --dry-run=client -o yaml | kubectl apply
 ↪  -f -

# Chapter 0.4 created hf-token-secret in the platform namespace
 ↪  (dynamo-system) — every
# DynamoGraphDeployment template also expects it in its own workload
 ↪  namespace, so re-create it
# here (repeat this for every other namespace you use in Chapters 1-9):
kubectl create secret generic hf-token-secret \
  --from-literal=HF_TOKEN=${HF_TOKEN} \
  -n ${NAMESPACE}

kubectl apply -f examples/backends/vllm/deploy/xpu/agg_xpu_dra.yaml -n
 ↪  ${NAMESPACE}
```

Verification:

```
kubectl get resourceclaim -n ${NAMESPACE}
kubectl get resourceslices
kubectl get dynamographdeployment -n ${NAMESPACE}
kubectl get pods -n ${NAMESPACE}
```

Inference Test:

```
kubectl port-forward deployment/vllm-agg-xpu-dra-frontend 8000:8000 -n
 ↪  ${NAMESPACE}
curl localhost:8000/v1/models
curl localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen/Qwen3-0.6B","prompt":"Hello","max_tokens":20}'
```

Troubleshooting: - If the `VllmDecodeWorker` pod stays `Pending`, check `kubectl describe resourceclaim` — a common cause is the Intel resource driver not yet reporting available devices (`kubectl get resourceslices` should list your XPU nodes). - If the worker crashes on startup complaining about missing device permissions, see §0.5.

Cleanup:

```
kubectl delete -f examples/backends/vllm/deploy/xpu/agg_xpu_dra.yaml -n
 ↪  ${NAMESPACE}
```

Config Reference:

Field	Value	Notes
DynamoGraphDeployment name	`vllm-agg-xpu-dra`	
Services	`Frontend` (1 replica), `VllmDecodeWorker` (1 replica, aggregated)	
Model	`Qwen/Qwen3-0.6B`	override via `--model` arg
`VLLM_TARGET_DEVICE`	`xpu`	

Field	Value	Notes
GPU claim	1× `gpu.intel.com` per worker replica	
`ephemeral-storage` request	`2Gi`	increase for larger models

1.2 Aggregated + Tracing (`agg_tracing_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: Same aggregated topology as §1.1, plus OpenTelemetry tracing instrumentation. Adds `DYN_LOGGING_JSONL=true` and `OTEL_EXPORT_ENABLED=true` at the `DynamoGraphDeployment` level, and per-service `OTEL_SERVICE_NAME` env vars (`dynamo-frontend`, `dynamo-worker-vllm`).

Prerequisites: Chapter 0 complete, plus an OpenTelemetry collector reachable from the cluster (the template's default `OTEL_EXPORTER_OTLP_TRACES_ENDPOINT` points at `http://tempo.observability.svc.cluster.local:4317` — a Grafana Tempo instance in an `observability` namespace; update or remove this if you use a different backend or haven't deployed one). See `docs/observability/tracing.md` in the repo for collector setup.

Deployment Steps:

```
# Edit OTEL_EXPORTER_OTLP_TRACES_ENDPOINT first if your collector endpoint
 ↪  differs
kubectl apply -f examples/backends/vllm/deploy/xpu/agg_tracing_xpu_dra.yaml -n
 ↪  ${NAMESPACE}
```

Verification / Inference Test: same as §1.1, but against Deployment `vllm-agg-tracing-xpu-frontend`. Additionally confirm traces are arriving at your collector (e.g. search for service name `dynamo-worker-vllm` in Tempo/Jaeger UI).

Troubleshooting: - No traces appearing: confirm `OTEL_EXPORTER_OTLP_TRACES_ENDPOINT` is reachable from the pod's network namespace (`kubectl exec` into the pod and `curl`/`nc` the endpoint). - If you have no observability stack deployed, remove the tracing env vars rather than pointing at a non-existent endpoint, to avoid startup delays from repeated failed export attempts.

Cleanup: `kubectl delete -f .../agg_tracing_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
`DynamoGraphDeployment` name	`vllm-agg-tracing-xpu`	
Extra env (deployment-level)	`DYN_LOGGING_JSONL=true`, `OTEL_EXPORT_ENABLED=true`, `OTEL_EXPORTER_OTLP_TRACES_ENDPOINT`	
Extra env (Frontend)	`OTEL_SERVICE_NAME=dynamo-frontend`	
Extra env (Worker)	`OTEL_SERVICE_NAME=dynamo-worker-vllm`	

1.3 Aggregated + KV Router (`agg_router_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: Adds Dynamo's KV-aware Router (`DYN_ROUTER_MODE=kv` on the Frontend) in front of 2 aggregated worker replicas. **Important upstream-documented limitation**: in Aggregated mode, `VllmDecodeWorker` does not emit `BlockStored` KV cache events (those are only produced during the prefill phase in Disaggregated mode), so the KV Router **cannot** make cache-aware routing decisions here — it effectively falls back to simpler load-based routing. For genuine KV-aware routing, use §1.8 (`disagg_router_xpu_dra.yaml`) instead. This case is documented for completeness, but readers who actually need KV-aware routing should go straight to §1.8.

Prerequisites: Chapter 0 complete.

Deployment Steps:

```
kubectl apply -f examples/backends/vllm/deploy/xpu/agg_router_xpu_dra.yaml -n
↪  ${NAMESPACE}
```

Verification / Inference Test: same pattern as §1.1, Deployment name `xpu-agg-router-frontend`, 2 worker replicas.

Troubleshooting: If you expected KV-cache-aware load balancing and don't see it, this is expected behavior per the limitation above — not a bug.

Cleanup: `kubectl delete -f .../agg_router_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
`DynamoGraphDeployment` name	`xpu-agg-router`	
`DYN_ROUTER_MODE`	kv (Frontend)	limited effectiveness in aggregated mode, see above
Worker replicas	2	
`--kv-events-config`	ZMQ publisher on `tcp://*:20080`, topic `kv-events`	
`--enable-prefix-caching`	set	

1.4 Aggregated + KV Router, No-KV-Events Mode (`agg_router_kv_app`

Status: documented from repo, not yet run hands-on.

Overview: A variant of §1.3 using `--no-kv-events` on the Frontend (`dynamo.frontend --router-mode kv --no-kv-events`) and `enable_kv_cache_events: false` on the worker. Instead of consuming the ZMQ-published KV events that §1.3 uses, the router predicts cache state locally from its own routing decisions with TTL-based expiration/pruning. The template's own comments describe this as avoiding a "NATS or JetStream" dependency — that appears to refer to an alternate/older KV-event distribution path, since §1.3's actual `--kv-events-config` uses a ZMQ publisher, not NATS; either way, this mode has no external message-bus dependency of its own. Benefit: simpler deployment for lightweight scenarios.

Prerequisites: Chapter 0 complete. No message-bus dependency needed for this variant.

Deployment Steps:

```
kubectl apply -f
 ↪  examples/backends/vllm/deploy/xpu/agg_router_kv_approx_xpu_dra.yaml -n
 ↪  ${NAMESPACE}
```

Verification / Inference Test: same pattern as §1.1, Deployment name `xpu-agg-kvap-frontend`.

Troubleshooting: Because routing is based on local prediction rather than real KV events, expect the router's cache-hit assumptions to drift under high churn — this is an inherent trade-off of the approximate mode, not a misconfiguration.

Cleanup: `kubectl delete -f .../agg_router_kv_approx_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
DynamoGraphDeployment name	xpu-agg-kvap	
Frontend command	dynamo.frontend --router-mode kv --no-kv-events	
--kv-events-config	{"enable_kv_cache_events": false}	

1.5 Disaggregated (disagg_xpu_dra.yaml) — Deep-Dive Sample Case

Status: documented from repo, not yet run hands-on.

Overview: True Prefill/Decode disaggregation on Intel XPU. Two distinct worker roles (`VllmPrefillWorker`, `VllmDecodeWorker`), each with `subComponentType: prefill/decode`, and KV cache handed off between them over NIXL (`kv_connector: NixlConnector`) with `kv_buffer_device: xpu` (i.e., the KV transfer buffer itself lives in XPU device memory, not host/CPU RAM).

Prerequisites: Chapter 0 complete. Both prefill and decode workers need their own XPU allocation (2 total GPU claims for this template as shipped, since both replicas default to 1).

Deployment Steps:

```
kubectl apply -f examples/backends/vllm/deploy/xpu/disagg_xpu_dra.yaml -n
 ↪  ${NAMESPACE}
```

Verification:

```
kubectl get resourceclaim -n ${NAMESPACE}       # expect 2 claims (prefill +
 ↪  decode)
kubectl get dynamographdeployment -n ${NAMESPACE}
kubectl get pods -n ${NAMESPACE}
kubectl logs deployment/vllm-disagg-xpu-dra-vllmprefillworker -n ${NAMESPACE}
kubectl logs deployment/vllm-disagg-xpu-dra-vllmdecodeworker -n ${NAMESPACE}
```

Inference Test:

```
kubectl port-forward deployment/vllm-disagg-xpu-dra-frontend 8000:8000 -n
 ↪  ${NAMESPACE}
```

```
curl localhost:8000/v1/models
curl localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"Qwen/Qwen3-0.6B","prompt":"Explain prefill/decode
   ↪  disaggregation","max_tokens":50}'
```

Troubleshooting: - KV transfer failures between prefill and decode workers typically show up as NIXL connection errors in the decode worker's log — confirm both workers were scheduled with XPU devices (`kubectl describe resourceclaim`) and that `kv_buffer_device: xpu` is set identically on both sides (`kv_role` itself differs by design: `kv_both` on prefill, `kv_consumer` on decode). - If either worker Pending forever, check `kubectl get resourceslices` for available device count — you need at least 2 free XPU devices across the cluster for this template.

Cleanup: `kubectl delete -f examples/backends/vllm/deploy/xpu/disagg_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
DynamoGraphDeployment name	`vllm-disagg-xpu-dra`	
Services	`Frontend`, `VllmPrefillWorker`, `VllmDecodeWorker` (1 replica each)	
`kv_connector`	`NixlConnector`	
Prefill `kv_role`	`kv_both`	
Decode `kv_role`	`kv_consumer`	
`kv_buffer_device`	`xpu`	KV buffer lives in XPU memory, not CPU RAM
`--block-size`	64	
Total GPU claims	2 (1 prefill + 1 decode)	

1.6 Disaggregated + Tracing (`disagg_tracing_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: Same disaggregated topology as §1.5, plus the same OpenTelemetry tracing pattern as §1.2 — `DYN_LOGGING_JSONL`/`OTEL_EXPORT_ENABLED` at the deployment level, and per-service `OTEL_SERVICE_NAME` (`dynamo-frontend`, `dynamo-worker-decode`, `dynamo-worker-prefill`).

Prerequisites: Chapter 0 complete + observability collector as in §1.2.

Deployment Steps:

```
kubectl apply -f examples/backends/vllm/deploy/xpu/disagg_tracing_xpu_dra.yaml
 ↪  -n ${NAMESPACE}
```

Verification / Inference Test: same pattern as §1.5, Deployment name `vllm-disagg-tracing-xpu-frontend`.

Troubleshooting: same as §1.2 (tracing endpoint reachability) and §1.5 (KV transfer).

Cleanup: `kubectl delete -f .../disagg_tracing_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference: identical to §1.5 plus the tracing env vars from §1.2, applied per-role (`dynamo-worker-decode` / `dynamo-worker-prefill` service names instead of a single `dynamo-worker-vllm`).

1.7 Disaggregated + Planner (`disagg_planner_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: Adds a `Planner` component that consumes pre-profiled throughput/latency curves (shipped as a `ConfigMap` named `planner-profile-data`, containing `prefill_raw_data.json` and `decode_raw_data.json`) to make scaling decisions. The template ships **example** profiling data points (ISL/TTFT/throughput-per-GPU for prefill; KV-usage/context-length/ITL/throughput for decode) — these are illustrative sample curves, not measured on real Intel XPU hardware, and must be replaced with your own profiling results before using the Planner for real capacity decisions (see `dynamo profile` tooling in the repo, if targeting production use).

Prerequisites: Chapter 0 complete, plus (recommended) your own hardware-profiled `prefill_raw_data.json`/`decode_raw_data.json` if you intend to trust the Planner's scaling recommendations rather than just exercising the deployment mechanics.

Deployment Steps:

```
kubectl apply -f examples/backends/vllm/deploy/xpu/disagg_planner_xpu_dra.yaml
 ↪  -n ${NAMESPACE}
```

Verification:

```
kubectl get configmap planner-profile-data -n ${NAMESPACE}
kubectl get pods -n ${NAMESPACE}     # expect Frontend, Planner,
 ↪  VllmDecodeWorker, VllmPrefillWorker
kubectl logs deployment/vllm-disagg-planner-xpu-planner -n ${NAMESPACE}
```

Inference Test: same as §1.5, Deployment name `vllm-disagg-planner-xpu-frontend`.

Troubleshooting: - The Planner's `--config` JSON hard-codes `prefill_engine_num_gpu: 1` and `decode_engine_num_gpu: 1` — the upstream comment flags this as a "KEY FIX: Add GPU counts here for DRA fallback" workaround, because the Planner cannot auto-discover GPU counts from DRA `ResourceClaims` the way it can from `resources.limits.gpu` on NVIDIA. If you change replica counts or TP size, update these values manually or the Planner's scaling math will be wrong. - `throughput_adjustment_interval_seconds: 60` controls how often the Planner re-evaluates scaling — reduce for faster reaction in a demo, increase to avoid thrashing in production.

Cleanup: `kubectl delete -f examples/backends/vllm/deploy/xpu/disagg_planner_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
`DynamoGraphDeployment` name	`vllm-disagg-planner-xpu`	
Extra component	`Planner` (`dynamo.planner`, `dynamo-planner:my-tag` image)	CPU-only, no XPU claim
`prefill_engine_num_gpu` / `decode_engine_num_gpu`	1 / 1	manual DRA fallback values — must match actual TP size

Field	Value	Notes
Profile data	`ConfigMap/planner-profile-data`	replace sample curves with real Intel XPU profiling data
`throughput_adjustment_interval_seconds`	60	

1.8 Disaggregated + KV Router (`disagg_router_xpu_dra.yaml`)

Status: documented from repo, not yet run hands-on.

Overview: The combination that upstream explicitly recommends for genuine KV-aware routing on Intel XPU — disaggregated P/D (so real `BlockStored` KV events are emitted during prefill) plus `DYN_ROUTER_MODE=kv` on the Frontend, `--kv-events-config` on prefill/decode workers, and `--enable-prefix-caching`. 2 decode worker replicas by default (vs. 1 for the plain disagg case in §1.5) to give the router something to route between.

Prerequisites: Chapter 0 complete.

Deployment Steps:

```
kubectl apply -f examples/backends/vllm/deploy/xpu/disagg_router_xpu_dra.yaml
↪  -n ${NAMESPACE}
```

Verification:

```
kubectl get pods -n ${NAMESPACE}   # expect Frontend, 2x VllmDecodeWorker,
↪  VllmPrefillWorker
kubectl get resourceclaim -n ${NAMESPACE}   # expect 3 claims total
```

Inference Test: same as §1.5, Deployment name `xpu-disagg-router-frontend`. Send several concurrent requests with overlapping prompt prefixes and observe (via logs/metrics) that requests sharing a KV prefix tend to route to the same decode worker.

Troubleshooting: same KV-transfer and DRA-scheduling checks as §1.5; additionally, if routing doesn't appear cache-aware, confirm `--kv-events-config` is actually enabled on the **prefill** worker (KV events are emitted during prefill, not decode, per the note in §1.3).

Cleanup: `kubectl delete -f examples/backends/vllm/deploy/xpu/disagg_router_xpu_dra.yaml -n ${NAMESPACE}`

Config Reference:

Field	Value	Notes
`DynamoGraphDeployment` name	`xpu-disagg-router`	
`DYN_ROUTER_MODE`	kv (Frontend)	fully effective here (unlike §1.3), since prefill emits real KV events
Decode worker replicas	2	
Total GPU claims	3 (2 decode + 1 prefill)	

Chapter 2 — Global Planner on Intel XPU

Status: documented from repo, not yet run hands-on.

Source: `examples/global_planner/global-planner-vllm-test-xpu-dra.yaml`, documented in `examples/global_planner/README.md`.

Overview: A single-endpoint, multi-pool deployment pattern: one `Frontend`, a `GlobalRouter`, and a `GlobalPlanner`, fronting **2 TP1 prefill pools + 1 TP1 decode pool** on Intel XPU (the XPU/DRA variant uses TP1 everywhere because each DRA `ResourceClaimTemplate` requests exactly one XPU device per worker — there is no multi-GPU-per-claim tensor-parallel config in this template). This differs from Chapter 1's simpler single-DGD templates by demonstrating Dynamo's cluster-wide GPU budget / multi-pool planning capability rather than a single fixed topology.

Prerequisites: Chapter 0 complete, plus an **RWX (ReadWriteMany) StorageClass** available in the cluster (`STORAGE_CLASS_NAME` env var below) for shared state between planner components.

Deployment Steps:

```
export K8S_NAMESPACE=my-ns
export DYNAMO_IMAGE=<dynamo-image>
export DYNAMO_VLLM_IMAGE=<vllm-xpu-image>   # from §0.3, must be built for XPU
export MODEL_NAME=Qwen/Qwen3-0.6B
export STORAGE_CLASS_NAME=<rwx-storage-class>

kubectl create secret generic hf-token-secret \
  --from-literal=HF_TOKEN=<your-token> -n ${K8S_NAMESPACE}

envsubst < examples/global_planner/global-planner-vllm-test-xpu-dra.yaml \
  | kubectl apply -n ${K8S_NAMESPACE} -f -
```

Verification:

```
kubectl get dynamographdeployment -n ${K8S_NAMESPACE}
kubectl get pods -n ${K8S_NAMESPACE}
kubectl get resourceclaim -n ${K8S_NAMESPACE}
```

Inference Test: port-forward the Frontend Deployment (`gp-ctrl-frontend`, following the `{DynamoGraphDeployment name}-frontend` pattern from §1.1) and send completion requests as in §1.1:

```
kubectl port-forward deployment/gp-ctrl-frontend 8000:8000 -n ${K8S_NAMESPACE}
curl localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d "{\"model\":\"${MODEL_NAME}\",\"prompt\":\"Hello\",\"max_tokens\":20}"
```

Observe (via `GlobalRouter`/`GlobalPlanner` logs) how requests are distributed across the 2 prefill pools and 1 decode pool.

Troubleshooting: - `envsubst` silently leaves variables unset if you forget to `export` one of them — always echo the rendered YAML or diff against the template before applying if a field looks empty. - Because every pool is TP1, do not attempt to scale a single pool's tensor-parallel size without first re-checking whether the DRA `ResourceClaimTemplate` needs to request more than 1 device — the shipped template does not do this.

Cleanup:

```
envsubst < examples/global_planner/global-planner-vllm-test-xpu-dra.yaml \
  | kubectl delete -n ${K8S_NAMESPACE} -f -
```

Config Reference:

Field	Value	Notes
Pattern	Single-endpoint, multi-pool	Frontend + GlobalRouter + GlobalPlanner
Prefill pools	2× TP1	XPU/DRA variant, 1 device per pool
Decode pool	1× TP1	
Required extra	RWX `StorageClass`	for planner shared state
Key difference from GPU (NVIDIA) variant	DRA `ResourceClaimTemplate` instead of `resources.limits.gpu`; `VLLM_TARGET_DEVICE=xpu`; TP1-only pools	

Chapter 3 — Recipe: Qwen3-VL-32B-FP8 Heterogeneous Hardware Disaggregation (Intel XPU Encode + NVIDIA GPU Decode) (In Progress)

Status: documented from official repo sources, not yet run hands-on.

Overview

`recipes/qwen3-vl-32b-fp8/` is a production-recipe deployment (distinct from the `examples/` tree) for **Qwen/Qwen3-VL-32B-Instruct-FP8**, a 32B FP8-quantized vision-language model. It ships two configurations:

Configuration	Hardware	Mode
`vllm/agg/`	1× NVIDIA H100/H200	Aggregated (vision + decode combined) — NVIDIA only, no Intel XPU
`vllm/hetero_hardware_disagg/`	**1× Intel XPU + 1× NVIDIA GPU**	Disaggregated — **this is the Intel XPU case**

Unlike every other Intel XPU case in this whitepaper (the 8 DRA templates in Chapter 1, which run **entirely** on Intel XPU), this recipe is **heterogeneous**: it deliberately splits the pipeline across two different accelerator vendors in the same deployment —

- **EncodeWorker** — runs the vision-encoding stage on **Intel XPU** (`gpu.intel.com` DRA device class), producing image embeddings.
- **VllmDecodeWorker** — runs autoregressive text decode on **NVIDIA GPU** (`gpu.nvidia.com` DRA device class), consuming those embeddings.
- Embeddings are transferred between the two workers over **RDMA** via a NIXL `NixlConnector` (`kv_buffer_device: xpu` on the encode side, `kv_buffer_device: cuda` on the decode side), using `UCX_TLS` that includes both `ze_copy` (Intel Level-Zero copy transport) and `cuda_copy`.

Source: `recipes/qwen3-vl-32b-fp8/README.md`, `vllm/hetero_hardware_disagg/deploy.yaml`, `vllm/hetero_hardware_disagg/intel_xpu_rdma_template.yaml`, `vllm/hetero_hardware_disagg/nvidi`

Why this differs from Chapter 1's DRA templates

	Chapter 1 (8 DRA templates)	This recipe
Location	`examples/backends/vllm/deploy/xpu/`	`recipes/qwen3-vl-32b-fp8/vllm/hetero_hardware_disagg/`
Hardware	100% Intel XPU	Intel XPU (encode) + NVIDIA GPU (decode) in the same deployment
Purpose	General serving patterns (aggregated/disaggregated × tracing/KV-router/planner) for any model	A specific production recipe for one model (Qwen3-VL-32B-FP8), demonstrating cross-vendor hardware disaggregation
DRA device class	`gpu.intel.com` only	`gpu.intel.com` (encode) + `gpu.nvidia.com` (decode), both requesting `rdma-dranet`

If your goal is "run Dynamo entirely on Intel XPU," use Chapter 1. If your goal is "offload multimodal encode to Intel XPU while keeping decode on existing NVIDIA GPU capacity" (e.g. to free up NVIDIA GPU capacity for decode-bound workloads while reusing available Intel XPU nodes for the comparatively lighter encode stage), this recipe is the direct reference.

Prerequisites

- Dynamo Platform installed (see Common Installation).
- A cluster with **both** an Intel XPU node/pool and an NVIDIA GPU node/pool, each exposed via Kubernetes DRA (`gpu.intel.com` and `gpu.nvidia.com` `DeviceClasses` respectively). The Intel side is the same Intel GPU DRA driver installed in Chapter 0; the NVIDIA side is a separate vendor-provided component, `kubernetes-sigs/dra-driver-nvidia-gpu` (install instructions on its documentation site), not covered elsewhere in this whitepaper since every other chapter is Intel XPU-only.
- **RDMA-capable network interfaces** between the Intel XPU and NVIDIA GPU nodes — this is a hard requirement, not optional; the recipe explicitly depends on a `rdma-dranet` DRA device class on both sides for the NIXL embedding transfer, provided by `kubernetes-sigs/dranet` (also not installed by either the Intel or NVIDIA GPU DRA drivers above — a third, separate component).
- A HuggingFace token with access to Qwen models, stored as `hf-token-secret`.
- A `StorageClass` supporting `ReadWriteMany` for the shared model cache PVC.

Deployment Steps

```
export NAMESPACE=dynamo-demo
export REPO_ROOT=$(realpath $(git rev-parse --show-toplevel))
cd ${REPO_ROOT}/recipes/qwen3-vl-32b-fp8   # all relative paths below are from
 ↪  here
kubectl create namespace ${NAMESPACE}

kubectl create secret generic hf-token-secret \
  --from-literal=HF_TOKEN="<your-token>" \
  -n ${NAMESPACE}
```

```
# 1. Update storageClassName in model-cache/model-cache.yaml to match your
 ↪  cluster, then:
kubectl apply -f model-cache/ -n ${NAMESPACE}
kubectl wait --for=condition=Complete job/model-download -n ${NAMESPACE}
 ↪  --timeout=3600s

# 2. Apply the DRA ResourceClaimTemplates (both, before the deployment)
kubectl apply -f vllm/hetero_hardware_disagg/intel_xpu_rdma_template.yaml -n
 ↪  ${NAMESPACE}
kubectl apply -f vllm/hetero_hardware_disagg/nvidia_gpu_rdma_template.yaml -n
 ↪  ${NAMESPACE}

# 3. Deploy
kubectl apply -f vllm/hetero_hardware_disagg/deploy.yaml -n ${NAMESPACE}
```

Verification

```
kubectl get pods -n ${NAMESPACE} -l
 ↪  nvidia.com/dynamo-graph-deployment-name=qwen3-vl-32b-fp8-vllm-disagg
kubectl get resourceclaims -n ${NAMESPACE}
kubectl logs -n ${NAMESPACE} deploy/qwen3-vl-32b-fp8-vllm-disagg-encodeworker
 ↪  | grep -i "nixl\|xpu"
kubectl logs -n ${NAMESPACE}
 ↪  deploy/qwen3-vl-32b-fp8-vllm-disagg-vllmdecodeworker | grep -i
 ↪  "nixl\|cuda"
```

Inference Test

```
kubectl port-forward svc/qwen3-vl-32b-fp8-vllm-disagg-frontend 8000:8000 -n
 ↪  ${NAMESPACE}

# Text-only request
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Qwen/Qwen3-VL-32B-Instruct-FP8",
    "messages": [{"role": "user", "content": "Hello!"}],
    "max_tokens": 50
  }'

# Multimodal (image) request — exercises the Intel XPU encode path
curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Qwen/Qwen3-VL-32B-Instruct-FP8",
    "messages": [
      {
        "role": "user",
        "content": [
          {"type": "image_url", "image_url": {"url":
 ↪  "https://upload.wikimedia.org/wikipedia/commons/thumb/4/47/PNG_transparency_demonstrat
 ↪  PNG_transparency_demonstration_1.png"}},
          {"type": "text", "text": "Describe this image in detail."}
        ]
```

```
      }
    ],
    "max_tokens": 256
  }'
```

Benchmarking

```
./benchmark/run-benchmark.sh --config disagg -n ${NAMESPACE}
```

Troubleshooting

- **`ResourceClaimTemplate` not found errors on deploy**: both `intel_xpu_rdma_template.yaml` and `nvidia_gpu_rdma_template.yaml` must be applied *before* `deploy.yaml` — the `deploy.yaml` references both templates by name (`intel-xpu-rdma-template`, `nvidia-gpu-rdma-template`).
- **NIXL handshake failures between encode and decode workers**: confirm RDMA connectivity between the two nodes independently of Dynamo first (e.g. `ib_write_bw`/`ucx_perftest`), and check that `UCX_TLS` includes the transport for each side (`ze_copy` for Intel XPU, `cuda_copy` for NVIDIA). `kv_connector_extra_config.enforce_handshake_compat: false` is already set to relax version matching between the two heterogeneous vLLM builds.
- **EncodeWorker OOM / low throughput**: `--gpu-memory-utilization 0.7` on the encode side is conservative — adjust based on the Intel XPU device's actual HBM/VRAM size.
- **Model download stuck**: confirm `storageClassName` supports `ReadWriteMany` and the PVC is `Bound`, not `Pending`.

Cleanup

```
kubectl delete -f vllm/hetero_hardware_disagg/deploy.yaml -n ${NAMESPACE}
kubectl delete -f vllm/hetero_hardware_disagg/intel_xpu_rdma_template.yaml -n
 ↪  ${NAMESPACE}
kubectl delete -f vllm/hetero_hardware_disagg/nvidia_gpu_rdma_template.yaml -n
 ↪  ${NAMESPACE}
kubectl delete -f model-cache/ -n ${NAMESPACE}
kubectl delete secret hf-token-secret -n ${NAMESPACE}
```

Config Reference

Field	Value	Notes
Model	`Qwen/Qwen3-VL-32B-Instruct-FP8`	32B params, FP8-quantized weights and KV cache
Encode image	`nvcr.io/nvidia/ai-dynamo/vllm-runtime-xpu:1.2.0`	Intel XPU vLLM runtime
Decode image	`nvcr.io/nvidia/ai-dynamo/vllm-runtime:1.2.0`	standard (NVIDIA) vLLM runtime
Intel XPU DRA device class	`gpu.intel.com`	requested alongside `rdma-dranet` in `intel-xpu-rdma-template`

Field	Value	Notes
NVIDIA GPU DRA device class	`gpu.nvidia.com`	requested alongside `rdma-dranet` in `nvidia-gpu-rdma-template`
Embedding transfer	NIXL `NixlConnector`, `DYN_VLLM_EMBEDDING_TRANSFER_MODE=read`	encode `kv_buffer_device: xpu`, decode `kv_buffer_device: cuda`
Encode-side UCX_TLS	`ib,rc,ud,rc_verbs,ud_verbs,ze_copy`	`ze_copy` = Intel Level-Zero copy transport
Decode-side UCX_TLS	`ib,rc,ud,rc_verbs,ud_verbs,cuda_copy`	
Decode-side KV cache dtype	`fp8`	
Shared memory per worker	80Gi	both Encode and Decode workers

Chapter 4 — Feasible but Not Yet Supported on Intel XPU

Answering “what could be supported but isn’t yet?”

Within `examples/backends/vllm/deploy/`, only 8 of the ~20 `DynamoGraphDeployment` templates have an Intel XPU (`xpu/`) counterpart (see Chapter 1). The other vLLM-backend templates below use the same vLLM engine already proven on Intel XPU, with no hardware-specific blocker found — these are gaps, not fundamental limitations:

Template	Path	Why it looks feasible on Intel XPU
LoRA adapter serving	`backends/vllm/deploy/lora/agg_lora.yaml`, `lora/multimodal/agg_qwen_lora.yaml`	Plain vLLM + CUDA-specific fields. Already confirmed working on Intel XPU via the bash launch-script path (`backends/vllm/launch/lora/xpu/agg_lo` — the K8s YAML gap is just a templating gap
GAIE (Gateway API Inference Extension) integration	`backends/vllm/deploy/gaie/{agg,disagg}.yaml` + `http-route.yaml`	Same disagg-layer pattern (EPP/InferencePool) already proven hardware-agnostic elsewhere
Multi-node disaggregation	`backends/vllm/deploy/disagg_multinode.yaml`	Same NIXL prefill/decode split as the XPU-proven `xpu/disagg_xpu_dra.yaml`, just spread across nodes

Template	Path	Why it looks feasible on Intel XPU
GPU Memory Service (GMS) sidecar / failover	`backends/vllm/deploy/agg_gms.yaml`, `agg_failover.yaml`, `gms-failover.yaml`	Kubernetes DRA itself is vendor-neutral (Intel already has its own DRA driver, used by all 8 Chapter 1 templates). The real dependency is GMS's own device backend — its design doc (`lib/gpu_memory_service/GMS_MULTI_DEV` shows CUDA support done and an Intel XPU backend planned as "Phase 2," just not implemented yet. Not a hard vendor lock-in, just unfinished
KVBM (multi-tier KV cache: GPU→CPU→SSD→remote)	`backends/vllm/deploy/agg_kvbm.yaml`, `disagg_kvbm*.yaml`	Intel XPU support is real, active work-in-progress, not yet merged: PRs #7946 and #10520 add SYCL/Level-Zero XPU support to KVBM v2, tracked by DEP issue #9313, still in design discussion
SGLang backend (all patterns)	`backends/sglang/deploy/{agg,agg_router,disagg,disagg_planner,dis multinode,agg_gms}.yaml`	No `xpu/` directory anywhere; SGLang engine's own Intel XPU support not independently verified — flagged as unconfirmed
Triton Server backend	`backends/tritonserver/`	No `xpu/` directory anywhere; NVIDIA Triton Inference Server has historically been NVIDIA-GPU-centric, but no explicit hard-blocker statement found in the repo — flagged as unconfirmed rather than a confirmed lock-in

None of these have been hands-on tested on Intel XPU — "feasible" here means "no repo evidence of a hardware blocker," not "verified working." Treat as a backlog of candidate work, not a completed case.

Genuinely NVIDIA-specific (real hardware/vendor lock-in found)

These have an explicit, documented dependency that does not currently have an Intel XPU equivalent:

Feature	Path	Lock-in reason
TensorRT-LLM backend (all patterns, incl. multimodal: Qwen3-VL, Llama4, LLaVA, Qwen2-VL)	`backends/trtllm/`	TensorRT-LLM is NVIDIA's proprietary compiler/runtime (TensorRT), fundamentally CUDA/NVIDIA-GPU-only — no Intel XPU port exists upstream in TensorRT-LLM itself, so this is a hard vendor lock-in, not a templating gap

Recipes (`recipes/`) other than the Chapter 3 hetero case

Confirmed via `grep -r xpu recipes/`: no other recipe under `recipes/` references Intel XPU as of this writing. Each recipe would need to be checked individually against the "feasible" criteria above (does it use vLLM/SGLang with no NVIDIA-only sidecar features) before assuming portability.

If you want to close these gaps

Building and validating any of the "feasible" candidates above on real Intel XPU hardware, then contributing the resulting `xpu/` template upstream, is the most direct way to expand real Intel XPU coverage — following the structural pattern of Chapter 1's existing 8 templates (Intel DRA device class `gpu.intel.com`, XCCL/`ze_copy` transport where NIXL/UCX is involved, custom `vllm-runtime-xpu` image).

Appendix A — Environment Variables Reference

Variable	Where set	Purpose
`VLLM_TARGET_DEVICE`	worker container	Must be `xpu` for every Intel XPU worker
`HF_TOKEN`	`hf-token-secret` (via `envFromSecret`)	HuggingFace model download auth
`DYN_ROUTER_MODE`	Frontend container	`kv` enables KV-aware routing
`DYN_LOGGING_JSONL`	deployment-level env	JSON-lines structured logging, used with tracing
`OTEL_EXPORT_ENABLED`	deployment-level env	Enables OpenTelemetry trace export
`OTEL_EXPORTER_OTLP_TRACES_ENDPOINT`	deployment-level env	OTLP collector endpoint (update per environment)
`OTEL_SERVICE_NAME`	per-service container env	Service name shown in trace UI

Appendix B — Known Issues / Workarounds

- **DRA GPU count not auto-discovered by Planner** (§1.7): the Planner cannot read GPU counts directly from `ResourceClaims` the way it reads `resources.limits.gpu` on NVIDIA. Manually set `prefill_engine_num_gpu`/`decode_engine_num_gpu` in the Planner's `--config` JSON to match your actual per-worker TP size, and keep them in sync whenever you change replica/TP configuration.

- **Aggregated mode + KV Router limitation** (§1.3): `VllmDecodeWorker` in aggregated mode does not emit `BlockStored` KV events (only produced during prefill in disaggregated mode), so KV Router falls back to non-cache-aware routing. Use §1.8 (`disagg_router_xpu_dra.yaml`) if genuine KV-aware routing matters for your use case.
- **Custom XPU image required for every worker** (§0.3): forgetting to build/push `vllm-runtime-xpu` (or leaving the placeholder tag `my-tag` unedited) is the most likely first-run failure — pods will `ImagePullBackOff` or `CrashLoopBackOff` immediately.
- **DRA API version mismatch**: all 8 templates use `apiVersion: resource.k8s.io/v1`. Clusters running older Kubernetes with only `resource.k8s.io/v1alpha3` or `v1beta1` DRA support will fail to apply these templates as-is; upgrade Kubernetes or check the Intel resource drivers' compatibility matrix.